

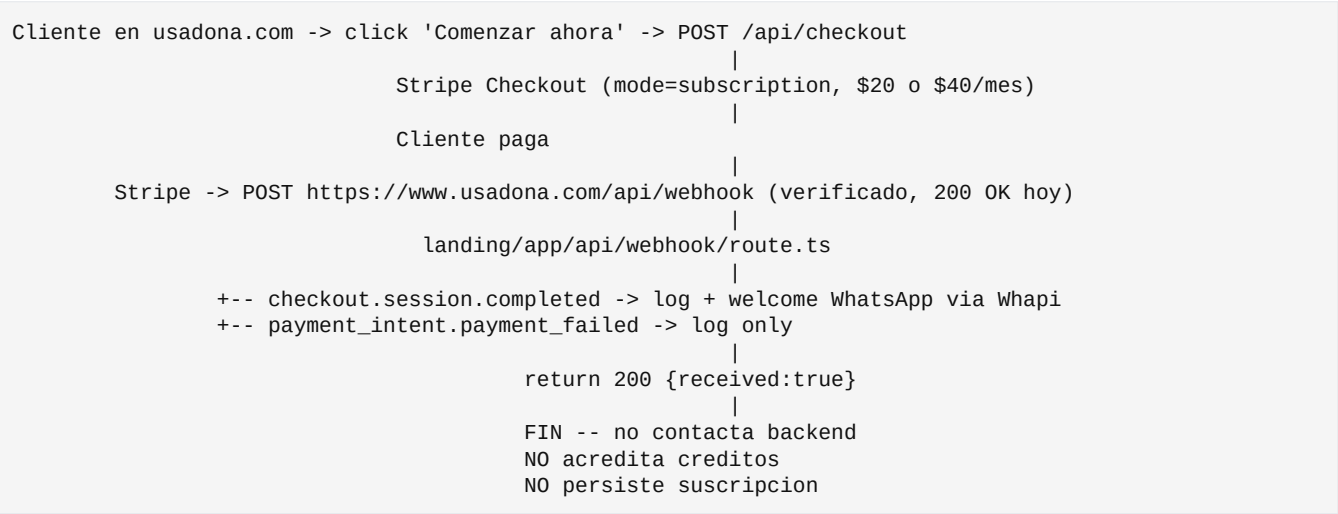
T1.3 — Stripe / backend de creditos como fuente unica

Diagnostico de solo lectura · Fecha: 2026-05-01 · Modo: solo lectura. · Sin cambios, sin commits, sin push.

Gap operativo critico: el cliente paga \$20/\$40 mensual via Stripe -> Stripe entrega webhook al landing -> landing manda welcome WhatsApp -> **el backend Dona nunca se entera**. Resultado: el sistema cobra pero no acredita creditos. T1.3 cierra el gap con un bridge landing -> backend autenticado por HMAC, sin cambiar Stripe Dashboard.

1. Diagnostico actual

1.1 Mapa de flujo actual



Backend /webhook/stripe: existe en agent/main.py:2122 con firma obligatoria (T0.2 mergeado), pero **no recibe trafico** porque Stripe Dashboard apunta al landing.

1.2 Estado del backend para creditos

Aspecto	Estado
agent/billing.py	Implementado para paquetes prepagos one-time (100/500/2000 creditos) con env vars STRIPE_PRICE_PAQUETE_*.
agent/billing.py:procesar_evento_stripe	Solo maneja checkout.session.completed y solo si metadata.creditos esta presente (formato paquete one-time).
agent/memory.py:SaldoCreditos	tabla simple con telefono (PK), saldo, total_comprado, total_consumido.
agent/memory.py:TransaccionCredito	audit log con idempotencia por stripe_session_id.
Tabla suscripciones	No existe. No hay SuscripcionStripe, Plan, customer_id ni subscription_id persistidos.
Manejo de invoice.* (renovaciones)	No implementado.

Aspecto	Estado
Manejo de subscription.{created, updated,deleted}	No implementado.

1.3 Modelo de identidad usuario

Lado	Llave
Backend	telefono E.164 (PK universal en todas las tablas).
Stripe	customer.id + subscription.id.
Landing	Sesion NextAuth amarrada al customer.id (no autenticada de verdad — tema separado de T0.1, no aplica aca).
Webhook landing	extrae phone desde session.metadata.phone o session.customer_details.phone.

Coincidencia: el phone del Stripe metadata es el mismo formato que el telefono del backend. Eso es bueno: misma llave, sin necesidad de tabla de mapping inicial.

1.4 Metadata de Stripe disponible hoy

landing/app/api/checkout/route.ts:41-45 setea en cada Checkout Session:

```
metadata: { plan, phone: phone || '' }
phone_number_collection: { enabled: true }
mode: 'subscription'
```

Lo que llega en el webhook checkout.session.completed:

Campo	Valor
session.metadata.plan	'premium' o 'pro'
session.metadata.phone	E.164 que el cliente puso en el form
session.customer_details.phone	E.164 que Stripe recolecto (puede diferir del anterior)
session.customer	cus_xxx
session.subscription	sub_xxx
session.id	cs_xxx (idempotencia)
session.metadata.creditos	NO existe — el handler backend espera esto y por eso retorna {handled: false, reason: 'metadata faltante'}

1.5 Backend recibe algun evento Stripe real hoy?

No. El webhook configurado en Stripe Dashboard es <https://www.usadona.com/api/webhook> (landing), no <https://dona-agent.onrender.com/webhook/stripe> (backend). El backend /webhook/stripe esta **listo y blindado pero sin trafico**.

2. Gap exacto

Capa	Estado actual	Lo que falta
Recepcion del webhook	Landing recibe Stripe ✓	Bridge landing -> backend (no existe)
Identificacion usuario	phone viaja en metadata ✓	Nada
Mapping plan -> creditos	No definido	Decision owner: cuantos creditos por plan
Persistencia suscripcion	No hay tabla	Crear tabla SuscripcionStripe
Acreditacion primer pago	Backend espera metadata.creditos (formato one-time)	Extender para mode=subscription
Renovaciones mensuales	No implementado	Manejar invoice.payment_succeeded
Cambio/cancelacion de plan	No implementado	Manejar customer.subscription.{updated,deleted}
Idempotencia eventos recurrentes	Idempotencia por session_id (sirve solo primera vez)	Idempotencia por event.id y/o invoice.id
Welcome WhatsApp	Landing lo manda ✓	Decidir si se mantiene en landing o se mueve a backend

El gap se resume asi: cliente paga \$20/\$40 mensual -> Stripe entrega webhook al landing -> landing manda welcome WhatsApp -> fin. El backend nunca se entera. El sistema de creditos sigue en cero hasta que alguien hace /admin/seed-creditos manual.

3. Propuesta de arquitectura minima

3.1 Decision de ruteo del webhook Stripe

Tres opciones evaluadas:

Opcion	Pro	Contra	Veredicto
A. Mantener Stripe -> landing, agregar bridge landing -> backend (HMAC entre ellos)	Cero cambio en Stripe Dashboard, transicion sin downtime	Dos hops, mantenimiento doble	Recomendada para Phase 1
B. Mover Stripe -> backend directo, deprecar landing webhook	Arquitectura limpia, una sola fuente	Requiere cambio en Stripe Dashboard, ventana de retries delicados	Recomendada para Phase 2 (despues de A)
C. Stripe -> ambos endpoints simultaneos	Cero downtime, ambos reciben	Doble procesamiento, duplicacion si idempotencia falla	Descartada

Recomendacion: Opcion A en este PR, dejar la posibilidad de B para despues.

3.2 Diagrama del flujo objetivo (Opcion A)

```
Cliente paga $20/$40
|
Stripe Checkout
|
```

```

Stripe -> POST https://www.usadona.com/api/webhook
|
landing/app/api/webhook/route.ts
| verifica firma Stripe
+--> welcome WhatsApp via Whapi (sigue igual)
+--> POST https://dona-agent.onrender.com/internal/stripe-event [NUEVO]
    + Header X-Internal-Signature (HMAC del body con shared secret)
    + Body: el evento Stripe verificado
        |
        agent/main.py:/internal/stripe-event [NUEVO endpoint]
        | verifica HMAC del landing
        | deduplica por event.id
        v
        agent/billing.py:procesar_evento_suscripcion(evento) [NUEVO]
        +- checkout.session.completed (mode=subscription)
        |   -> upsert SuscripcionStripe + acreditar creditos del mes
        +- invoice.payment_succeeded (renovacion)
        |   -> acreditar creditos del nuevo periodo (idempotente por invoice.id)
        +- customer.subscription.updated (cambio de plan)
        |   -> actualizar SuscripcionStripe (plan, status, creditos_mes)
        +- customer.subscription.deleted (cancelacion)
        |   -> marcar SuscripcionStripe.status='canceled', no acreditar mas

```

3.3 Por que un endpoint /internal/* y no reusar /webhook/stripe

- /webhook/stripe espera firma de Stripe directa. Si el landing reenvia, la firma original ya no aplica al body re-construido.
- /internal/stripe-event valida HMAC propio entre landing y backend con un secret nuevo (INTERNAL_BRIDGE_SECRET). El landing es el unico cliente legitimo.
- Mantener /webhook/stripe operativo permite migrar a Opcion B en el futuro sin tocar este endpoint.
- Sin doble proposito. Cada endpoint tiene una responsabilidad clara.

3.4 Idempotencia robusta

Capa	Llave	Proposito
Landing recibe Stripe	event.id	Stripe puede reentregar; landing no debe procesar dos veces.
Bridge landing -> backend	event.id	Si landing reenvia retry, backend no procesa dos veces.
Backend acredita primer pago	session.id (existente) o event.id	Lo que ya hace, vale.
Backend acredita renovacion	invoice.id	Cada renovacion tiene invoice.id unico. Nuevo .
Backend persiste suscripcion	subscription.id (PK)	Una sola fila por suscripcion, upsert.

4. Archivos que tocaria

4.1 Backend (agent/)

Archivo	Cambio
agent/memory.py	+ clase SuscripcionStripe con: telefono (FK logico), customer_id, subscription_id (PK), plan_codigo, price_id, status, creditos_mensuales, ultimo_invoice_acreditado, creado, actualizado. + clase EventoStripeProcesado con event_id (PK), tipo, recibido_en.
agent/billing.py	+ procesar_evento_suscripcion(evento) con handlers para los 4 tipos. + helper creditos_de_plan(price_id) que mapea env vars STRIPE_CREDITOS_PREMIUM/PRO a int. + _evento_ya_procesado(event_id) para idempotencia.
agent/main.py	+ endpoint POST /internal/stripe-event con verificacion HMAC del header X-Internal-Signature contra INTERNAL_BRIDGE_SECRET. + import top-level (mismo patron T0.2/T0.4).
.env.example	+ INTERNAL_BRIDGE_SECRET (obligatorio en prod). + STRIPE_CREDITOS_PREMIUM y STRIPE_CREDITOS_PRO (default sugerido pendiente de decision owner).
tests/test_billing.py	+ tests para procesar_evento_suscripcion con cada uno de los 4 tipos + idempotencia + plan desconocido.
tests/test_main_internal_stripe.py (nuevo)	+ tests del endpoint /internal/stripe-event: HMAC ausente/invalido/correcto, evento duplicado, evento bien procesado.
alembic/versions/00X_suscripcion_stripe.py (nuevo)	migracion para crear suscripcion_stripe y evento_stripe_procesado.

4.2 Landing (landing/)

Archivo	Cambio
landing/app/api/webhook/route.ts	Despues del welcome WhatsApp, hacer fetch(BACKEND_URL + '/internal/stripe-event', {...}) con HMAC. Si falla, loguear pero no bloquear (Stripe ya considero el evento OK). Idealmente: queue de reintentos (Phase 2).
landing/lib/internal-bridge.ts (nuevo)	helper para firmar y enviar al backend, con timeout corto.
landing/.env (NO TOCAR DESDE CODIGO)	Vercel env: INTERNAL_BRIDGE_SECRET (mismo valor que en Render) + BACKEND_URL=https://dona-agent.onrender.com.

Nota: el landing existente no requiere cambios disruptivos. El welcome WhatsApp sigue donde esta. Solo se agrega un fetch al final del handler.

4.3 Lo que NO toco en este PR

- /webhook/stripe directo en backend -> queda como esta (blindado, sin trafico).
- Stripe Dashboard URL -> no se modifica.
- Tablas existentes (SaldoCreditos, TransaccionCredito) -> sin alteraciones.
- Modelo de paquetes one-time prepagos -> sigue funcional, complementario al de suscripciones.

- Welcome WhatsApp -> sigue en landing por ahora.

5. Cambios de DB / migraciones

5.1 Tabla nueva suscripcion_stripe

```
class SuscripcionStripe(Base):
    __tablename__ = "suscripcion_stripe"

    subscription_id: Mapped[str] = mapped_column(String(200), primary_key=True)
    telefono: Mapped[str] = mapped_column(String(50), index=True)
    customer_id: Mapped[str] = mapped_column(String(200), index=True)
    plan_codigo: Mapped[str] = mapped_column(String(50)) # "premium" | "pro"
    price_id: Mapped[str] = mapped_column(String(200))
    status: Mapped[str] = mapped_column(String(40)) # active | past_due | canceled | incomplete

    creditos_mensuales: Mapped[int] = mapped_column(Integer)
    ultimo_invoice_acreditado: Mapped[str] = mapped_column(String(200), default="")
    creado: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    actualizado: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
```

5.2 Tabla nueva evento_stripe_procesado

```
class EventoStripeProcesado(Base):
    __tablename__ = "evento_stripe_procesado"

    event_id: Mapped[str] = mapped_column(String(200), primary_key=True)
    tipo: Mapped[str] = mapped_column(String(80), index=True)
    recibido_en: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
```

5.3 Migracion Alembic

alembic/versions/00X_suscripcion_stripe.py con upgrade() que crea ambas tablas y downgrade() que las dropea. Sin tocar 001_estado_inicial.py.

Nota critica: el repo aun usa metadata.create_all() en agent/main.py:lifespan (T4.3 esta pendiente). Eso significa que las tablas nuevas se crean automaticamente al arrancar si no existen — la migracion Alembic es idempotente y redundante con el create_all, pero conviene tenerla para tener un baseline registrado.

6. Tests necesarios

6.1 Backend

```
class TestProcesarEventoSuscripcion:
    def test_checkout_session_completed_subscription_acredita_y_persiste
    def test_checkout_session_completed_payment_legacy_sigue_funcionando # paquetes prepagos
    def test_invoice_payment_succeeded_acredita_creditos_del_periodo
    def test_invoice_payment_succeeded_idempotente_por_invoice_id
    def test_subscription_updated_cambia_plan_y_creditos_mensuales
    def test_subscription_deleted_marca_canceled_y_no_acredita_mas
    def test_evento_duplicado_por_event_id_se_ignora
    def test_plan_desconocido_no_acredita_y_loguea_warning
    def test_telefono_faltante_loguea_y_no_acredita

class TestInternalStripeEndpoint:
    def test_hmac_ausente_responde_401
    def test_hmac_invalido_responde_401
    def test_hmac_valido_procesa_evento
    def test_production_sin_INTERNAL_BRIDGE_SECRET_aborta_startup
    def test_evento_duplicado_responde_200_idempotente
```

Total estimado: ~**12-14 tests nuevos**.

6.2 Landing

El landing hoy no tiene suite de tests visible — eso es deuda separada. Para este PR podemos limitarnos a tests manuales del lado landing.

7. Riesgos

Riesgo	Probabilidad	Impacto	Mitigacion
Doble acreditacion si Stripe reentrega el webhook al landing	Baja	Alto	Idempotencia por event.id en backend (evento_stripe_procesado)
Doble acreditacion si landing reenvia al backend dos veces	Baja-media	Alto	Misma idempotencia
Perdida de eventos si fetch landing -> backend falla	Media	Medio	Stripe reintenta 3 veces si landing responde no-200; si landing siempre responde 200, se pierde. Mitigacion: landing responde 500 a Stripe si bridge falla (Stripe reintenta) o logea + responde 200 (cliente recibe welcome igual; bridge se completa por mecanismo manual). Decision owner.
Plan desconocido (price_id que no matchea Premium/Pro)	Baja	Medio	Loguear warning + no acreditar; alerta al admin.
Mismatch telefono entre metadata.phone y customer_details.phone	Media	Bajo	Preferir metadata.phone (form del cliente); fallback a customer_details.phone.
Subscription cambia de plan sin que recibamos el evento	Baja	Medio	Si llega el siguiente invoice.payment_succeeded, leemos price_id actual del Stripe customer para reconciliar.
Cliente paga pero no completa onboarding	Existente	Bajo	Welcome WhatsApp ya cubre eso. Backend acredita; cuando el cliente conecte por WhatsApp con el mismo numero, ve sus creditos.
Race condition entre primer checkout.session.completed y primer invoice.payment_succeeded (mismo periodo)	Baja	Medio	Acreditar solo en invoice.payment_succeeded y dejar checkout.session.completed para crear/actualizar la suscripcion. Mi voto: simpler asi.
Vercel env vars desincronizadas con Render env vars (mismo INTERNAL_BRIDGE_SECRET)	Media	Alto	Setear ambos juntos antes del merge. Documentar.

8. Plan de implementacion por PRs pequenos

PR T1.3.A — Modelos + migracion (backend solo)

- agent/memory.py: clases SuscripcionStripe, EventoStripeProcesado.
- alembic/versions/00X_suscripcion_stripe.py.
- Tests de modelo simple (insert/query).
- **Cero impacto en produccion**: solo crea tablas vacias al deploy.
- ~80 lineas.

PR T1.3.B — Helper `creditos_de_plan` + matrices (backend solo)

- `agent/billing.py`: helper que mapea `price_id` -> `creditos mensuales` segun `env vars STRIPE_CREDITOS_PREMIUM / STRIPE_CREDITOS_PRO`.
- `.env.example`: documentar las dos variables nuevas.
- Tests del helper.
- **Cero impacto en produccion** sin las `env vars` seteadas.
- ~30 lineas. **Bloqueador: decision owner sobre cuantos creditos.**

PR T1.3.C — procesar_evento_suscripcion + idempotencia (backend solo)

- `agent/billing.py`: nueva funcion con los 4 handlers.
- `agent/billing.py`: helper `_evento_ya_procesado(event_id)`.
- Tests exhaustivos.
- **Cero impacto en produccion** sin endpoint que la invoque.
- ~150 lineas (incluyendo tests).

PR T1.3.D — Endpoint `/internal/stripe-event` con HMAC (backend solo)

- `agent/main.py`: endpoint nuevo + verificacion HMAC + import top-level fail-fast en prod sin `INTERNAL_BRIDGE_SECRET`.
- `.env.example`: documentar `INTERNAL_BRIDGE_SECRET`.
- Tests del endpoint.
- **Cero impacto en produccion** mientras nadie lo invoque (landing aun no lo usa).
- ~100 lineas (incluyendo tests).

PR T1.3.E — Bridge landing -> backend (landing solo)

- `landing/app/api/webhook/route.ts`: agregar `fetch` al final del handler.
- `landing/lib/internal-bridge.ts`: helper de firma + envio.
- **Impacto en produccion**: ahora cada `checkout.session.completed` reenvia. Riesgo: si backend falla, decidir 200 o 500 a Stripe.
- **Pre-requisito**: setear `INTERNAL_BRIDGE_SECRET` en Vercel y Render con el mismo valor.
- ~40 lineas.

PR T1.3.F (opcional) — Mover Stripe Dashboard a backend directo

- Solo cuando T1.3.E haya estado estable por una semana.
- Cambiar URL en Stripe Dashboard a `https://dona-agent.onrender.com/webhook/stripe`.
- Adaptar `agent/billing.py:procesar_evento_stripe` para invocar `procesar_evento_suscripcion` cuando sea de `subscription`.
- Landing webhook se vuelve dead code -> deprecarse en housekeeping.

9. Checklist antes de merge / deploy

9.1 Pre-PR (decisiones bloqueantes del owner)

- [] **Cuantos credits por plan:** Premium \$20 -> ¿N credits/mes? Pro \$40 -> ¿M credits/mes? Recomendacion tentativa: Premium 100, Pro 500 (para empezar conservador).
- [] **Comportamiento ante fallo del bridge:** si backend no responde, ¿landing devuelve 200 a Stripe (cliente queda sin credits hasta reconcilio manual) o 500 (Stripe reintenta hasta 3 veces; tras eso cliente queda sin credits)?
- [] **Renovaciones:** ¿credits se acumulan mes a mes (cliente nunca pierde) o se resetean al inicio de cada periodo?
- [] **Cancelacion:** cuando subscription se cancela, ¿credits restantes se mantienen hasta consumir, o se borran al final del periodo pago?
- [] **Generar INTERNAL_BRIDGE_SECRET** con `python -c "import secrets; print(secrets.token_urlsafe(32))"`.

9.2 Pre-merge (operativo)

- [] STRIPE_CREDITOS_PREMIUM y STRIPE_CREDITOS_PRO seteados en Render.
- [] INTERNAL_BRIDGE_SECRET seteado tanto en **Vercel (landing)** como en **Render (backend)** con el **mismo valor**.
- [] Confirmar que la URL del backend desde Vercel es accesible (sin firewall ni IP allowlist).
- [] Decidir si se mantiene welcome WhatsApp en landing o se mueve a backend (recomendacion: mantener en landing por ahora, mover en T1.3.F).

9.3 Durante el PR (local)

- [] Implementar PR T1.3.A -> D en orden, mergeando uno por uno.
- [] Despues del .D mergeado y desplegado: implementar T1.3.E.
- [] `pytest -q` localmente verde en cada PR.
- [] Tests manuales con Stripe Test Mode antes de mergear T1.3.E.

9.4 Post-merge (verificacion)

- [] Hacer una compra real en **modo test** desde la landing.
- [] Verificar Render logs: '[BILLING] Acreditado N cr -> +XXX...' y '[INTERNAL] Evento procesado: event_id=evt_...'
- [] Verificar DB backend: fila en suscripcion_stripe + fila en transacciones_credito + saldo en saldo_creditos correcto.
- [] Verificar que el cliente recibe welcome WhatsApp Y que *dona saldo* por WhatsApp le devuelve los credits correctos.
- [] Esperar primera renovacion real (siguiente mes) y verificar que `invoice.payment_succeeded` acredita el segundo periodo.

10. Resumen ejecutivo

Aspecto	Detalle
Gap operativo	Cliente paga \$20/\$40 mensual -> backend Dona no acredita creditos -> producto cobra pero no entrega
Causa raiz	Stripe webhook va al landing, landing solo manda welcome, backend nunca se entera
Solucion minima	Bridge landing -> backend con HMAC + nuevas tablas + handlers de subscription/invoice events
Patron	Opcion A (mantener Stripe en landing, agregar bridge interno). Opcion B (mover Stripe al backend) queda para Phase 2
Archivos	Backend: 4 archivos (memory.py, billing.py, main.py, .env.example) + 2 tests + 1 migracion Alembic. Landing: 2 archivos (webhook/route.ts, lib/internal-bridge.ts)
DB	2 tablas nuevas: suscripcion_stripe, evento_stripe_procesado
Tests	~12-14 nuevos en backend
PRs	5 pequenos (A->E) en orden, opcional F despues
Decisiones bloqueantes del owner	4 (creditos por plan, comportamiento ante fallo bridge, acumulacion renovaciones, cancelacion)
Riesgo de deploy	Bajo si las env vars estan seteadas y los PRs van en orden
Welcome WhatsApp actual	Se mantiene en landing por simplicidad; se mueve a backend en T1.3.F (opcional)

Estado: no implementado todavia. Esperando OK del owner con respuesta a las 4 decisiones bloqueantes y autorizacion para arrancar con PR T1.3.A.